Faculty of Engineering and Technology
Electrical and Computer Engineering Department

Operating Systems
ENCS3390

Project 1 Report

……

Prepared by:
Lara Foqaha 1220071

Instructor: Abdel Salam Sayyad

Section: 3

Date: 8/12/2024

Abstract

This project explores the performance and efficiency of three distinct computational approaches—naive, multiprocessing, and multithreading—in analyzing the enwik8 dataset to identify the ten most frequently occurring words. Implemented in a controlled computing environment with at least four cores, the study evaluates execution times across varying numbers of child processes and threads (2, 4, 6, 8). Leveraging Amdahl's Law, the project assesses the impact of parallelism, identifying the serial portion of the code and calculating theoretical speedup limits. A comparative analysis highlights the trade-offs and benefits of each method, offering insights into optimal parallelism strategies for word frequency analysis tasks. The findings provide a practical understanding of performance tuning in computational systems, relevant to modern multi-core architectures.

Theory

Naïve Approach

The naive approach represents the simplest method for processing tasks, executing operations sequentially within a single process without utilizing any form of parallelism. While straightforward and easy to implement, this approach lacks the ability to exploit the capabilities of multi-core processors. In the context of this project, the naive approach processes the entire enwik8 dataset sequentially to determine the top 10 most frequent words. This method is computationally expensive for large datasets, as it does not leverage parallel execution to distribute the workload, resulting in increased processing time and suboptimal performance for intensive tasks.

Multiprocessing Approach

The multiprocessing approach involves splitting the workload across multiple processes, each running independently and on separate processor cores. In this project, the dataset is divided into chunks, with each chunk processed by a child process. This method utilizes inter-process communication mechanisms to aggregate the results from all child processes. Multiprocessing is highly effective for tasks that can be partitioned into independent subtasks, as it minimizes contention and leverages the full computational power of multi-core systems. However, it incurs additional overhead due to process creation and inter-process communication, especially when the number of processes increases.

Multithreading Approach

The multithreading approach enables parallel execution within a single process by creating multiple threads, each capable of executing subtasks concurrently. In this project, the dataset is divided into segments, with each segment handled by a separate thread. Threads share the same memory space, allowing for efficient data access and communication. This approach is particularly advantageous for tasks requiring fine-grained parallelism and frequent communication between subtasks. However, it introduces challenges such as synchronization overhead and potential issues like data races or deadlocks. Despite these complexities, multithreading typically incurs lower overhead than multiprocessing, making it well-suited for tasks with significant shared data access.

Description of the Environment

The experimental setup for this project is as follows:

- **Processor (CPU):** The system features an Intel 11th Gen Core i7-1165G7 processor with 4 physical cores and a base clock speed of 2.8 GHz.
- **Memory (RAM):** The computer is equipped with 7.7 GB of RAM, of which 6.6 GB was available during testing.
- **Storage:** An SSD was used for storage, enabling fast data access and read/write operations.
- **Operating System:** The project was conducted on the Windows operating system.
- **Programming Language:** The implementation was carried out in the C programming language.
- **Development Environment:** The project was developed using Visual Studio Code, a versatile and efficient IDE.
- **Virtualization:** No virtual machine was utilized, allowing the program to run directly on the physical hardware to maximize performance.

This environment provided a reliable platform for evaluating the naive, multiprocessing, and multithreading approaches, ensuring accurate measurements of computational efficiency and performance.

Achieving Requirements

Multiprocessing

To implement multiprocessing in this project, several APIs and functions were used to efficiently divide and manage the workload across multiple processes. The implementation is as follows:

1. **Process Creation and Management**:
    - The *fork()* function was utilized to create child processes. Each child process was responsible for processing a specific chunk of the input file.
    - *wait(NULL)* was used in the parent process to ensure that all child processes completed their execution before proceeding to the final aggregation step.
2. **Shared Memory for Inter-process Communication**:
    - *mmap()* with flags `MAP_SHARED` and `MAP_ANONYMOUS` was used to allocate shared memory for the `globalWords` array and the `globalSize` variable. This allowed all processes to share and update a common memory space, enabling efficient aggregation of word frequencies across processes.
    - The shared memory approach reduced the need for complex inter-process communication mechanisms, such as message passing or pipes.
3. **File Chunking and Distribution**:
    - The file was divided into chunks based on its size using *ftell()* and *fseek()*. Each child process processed a distinct chunk to ensure minimal overlap and redundancy.

    - A boundary adjustment mechanism was implemented to ensure that partial words at chunk edges were handled correctly.
4. **Local Processing and Aggregation**:
    - Each child process maintained its local data structure for storing words and their frequencies (`localWords` and `localSize`), which were later merged into the global shared memory using the `mergeResults` function.
    - **Dynamic memory allocation (`malloc`)** was used to manage memory for the local words array within each child process.
5. **Word Counting and Merging**:
    - The `countWordsInFile` function handled the word counting for the assigned chunk of the file. Words were stored in the local data structure, and their frequencies were updated if a match was found.
    - The `mergeResults` function merged the locally processed results into the global shared memory, ensuring that word frequencies were aggregated correctly across all processes.
6. **Synchronization**:
    - Implicit synchronization was achieved by ensuring that the parent process waited for all child processes to complete (`wait(NULL)`) before accessing the shared memory to sort and find the top 10 frequent words.

This approach efficiently leveraged multiprocessing to parallelize the task, taking advantage of the system's multi-core architecture to reduce execution time for processing the large dataset.

<u>Multithreading</u>

To implement multithreading for this project, the following approach and functionalities were utilized:

1. Thread Creation and Execution

The program uses the `pthread` library to create and manage threads. `pthread_create()` is used to spawn threads, each tasked with processing a specific chunk of the file. Each thread is assigned a byte range (start and end positions) of the file to ensure parallelism.

2. Data Passing to Threads

A `ThreadData` structure is defined to encapsulate necessary information for each thread. It includes:

- The thread's unique identifier.
- The start and end byte positions in the file.
- Shared pointers to the global word frequency array and its size.
- A mutex for synchronized access to shared resources.
- The file name to be processed.

3. File Division

The file is divided into equal chunks by calculating its size and dividing it by the number of threads (`NUM_THREADS`). Each thread processes a separate chunk based on its assigned range, ensuring efficient division of labor.

4. Word Counting

The `countWordsInFile` function is executed by each thread. It processes its assigned file chunk, skipping any partial words at the boundaries and maintaining a local array of word frequencies. The local results are then merged into the global shared word frequency array.

5. Synchronization

A `pthread_mutex_t` mutex is employed to ensure synchronized access to shared resources. When a thread updates the global word array, it locks the mutex using `pthread_mutex_lock()` to prevent other threads from interfering. Once the update is complete, the mutex is unlocked with `pthread_mutex_unlock()`.

6. Final Aggregation

After all threads finish execution, the main thread aggregates the results stored in the global word array and sorts it using the `quickSort` function. The sorted array is then used to extract the top 10 most frequent words.

This multithreading approach allows the workload to be evenly distributed across multiple threads, utilizing available CPU cores to improve processing efficiency.

Analysis According to Amdahl's Law

Multiprocessing

- **Percentage of the Serial Part of the Code**:

Based on the code, the serial part primarily includes:

- Setting up shared memory using `mmap`.
- File initialization and dividing the file into chunks (`ftell`, `fseek`).
- Waiting for all child processes using `wait(NULL)`.
- Merging results from the child processes into the shared memory (`mergeResults`).
- Sorting the words and finding the top 10 (`quickSort` and `findTop10`).

These operations do not benefit from parallel execution and constitute the serial portion of the program. Estimating conservatively, about **20-30% of the program** is serial, while the remaining 70-80% involves parallel tasks, such as counting words in chunks of the file using child processes.

- **Maximum Speedup According to the Available Number of Cores**:

Amdahl's Law states:

Speedup = 1 / (S + (P/N))

Where:

- S is the serial portion of the code.
- P is the parallel portion of the code.
- N is the number of processor cores available.

For a system with 4 cores:

Speedup = 1 / (0.3 + (0.7/4)) = 1 / 0.475 ≈ 2.11

For a system with 8 cores (hypothetically, if the hardware allowed):

Speedup = 1 / (0.3 + (0.7/8)) ≈ 2.58

This shows that increasing the number of cores provides diminishing returns due to the presence of the serial portion.

- **Optimal Number of Child Processes**:

Given that the system has 4 physical cores, the optimal number of child processes should ideally match the number of available cores to maximize CPU utilization. Beyond 4 child processes, performance might degrade due to:

- Overhead from context switching between processes.

- Contention for shared memory and CPU resources.

Therefore, **4 child processes** is optimal for this system. However, in the current implementation with 2 child processes (`NUM_PROCESSES = 2`), the system achieves a good balance between performance and simplicity, while not fully utilizing the available cores. Testing with 4 child processes could lead to further performance improvements.

Multithreading

1. Percentage of the Serial Part of the Code

The serial part of the program includes operations such as initialization, setting up threads, and processing the final aggregated results. These steps account for approximately **25%** of the overall execution time. The remaining **75%** of the workload is parallelizable.

2. Maximum Speedup

Using Amdahl's Law

$S = 1/ (S + (P/N))$

Where $S=0.25$, $P=0.75$, $N=4$

$S= 2.29$

The theoretical maximum speedup is **2.29x**.

3. Optimal Number of Threads

The analysis shows that increasing the number of threads improves performance up to the system's hardware limits. With four cores available, using four threads allows the program to utilize the processing power efficiently.

Comparing performances

Approach	2 children	4 children	6 children	8 children
Naïve	622.9 sec	-	-	-
Multiprocessing	455.1 sec	376.5 sec	271.6 sec	267.5 sec
Multithreading		504.2 sec	469.9 sec	461.9 sec

```
TERMINAL

Time taken to read from file: 435.267321 seconds
Time taken to sort: 19.884518 seconds
Top 10 most frequent words:
1. the - 1061396 times
2. of - 593677 times
3. and - 416629 times
4. one - 411764 times
5. in - 372201 times
6. a - 325873 times
7. to - 316376 times
8. zero - 264975 times
9. nine - 250430 times
10. two - 192644 times
Time taken to print words: 0.000006 seconds
Execution time: 455.151970 seconds
[1] + Done                    "/usr/bin/gdb" -
pykkrc.x23"
root@LaraPC:/home/os_project1#
```

```
TERMINAL

Time taken to read from file: 248.173264 seconds
Time taken to sort: 19.371204 seconds
Top 10 most frequent words:
1. the - 1061396 times
2. of - 593677 times
3. and - 416629 times
4. one - 411764 times
5. in - 372201 times
6. a - 325873 times
7. to - 316376 times
8. zero - 264975 times
9. nine - 250430 times
10. two - 192644 times
Time taken to print words: 0.000017 seconds
Execution time: 267.544712 seconds
[1] + Done                    "/usr/bin/gdb" -
iztsjb.1j3"
root@LaraPC:/home/os_project1#
```

```
TERMINAL

Time taken to read from file: 252.153442 seconds
Time taken to sort: 19.500781 seconds
Top 10 most frequent words:
1. the - 1061396 times
2. of - 593677 times
3. and - 416629 times
4. one - 411764 times
5. in - 372201 times
6. a - 325873 times
7. to - 316376 times
8. zero - 264975 times
9. nine - 250430 times
10. two - 192644 times
Time taken to print words: 0.000007 seconds
Execution time: 271.654434 seconds
[1] + Done                    "/usr/bin/gdb" --
w2px44.cia"
root@LaraPC:/home/os_project1#
```

```
TERMINAL

Time taken to read from file: 351.360336 seconds
Time taken to sort: 25.156650 seconds
Top 10 most frequent words:
1. the - 1061396 times
2. of - 593677 times
3. and - 416629 times
4. one - 411764 times
5. in - 372201 times
6. a - 325873 times
7. to - 316376 times
8. zero - 264975 times
9. nine - 250430 times
10. two - 192644 times
Time taken to print words: 0.000007 seconds
Execution time: 376.517215 seconds
[1] + Done                    "/usr/bin/gdb" -
e5xla1.bnr"
root@LaraPC:/home/os_project1#
```

```
TERMINAL

Time taken to read from file: 572.266945 seconds
Top 10 most frequent words:
1. kinkel - 2 times
2. anarchism - 2 times
3. originated - 2 times
4. as - 2 times
5. a - 2 times
6. term - 2 times
7. of - 2 times
8. abuse - 2 times
9. first - 2 times
10. used - 2 times
Execution time: 623.628871 seconds
[1] + Done                    "/usr/bin/gdb" -
cjksby.nlj"
root@LaraPC:/home/os_project1#
```

```
TERMINAL

Time taken to read from file: 469.006825 seconds
Top 10 most frequent words:
1. bellingshausen - 4 times
2. technology - 4 times
3. used - 4 times
4. to - 4 times
5. transfer - 4 times
6. copies - 4 times
7. of - 4 times
8. documents - 4 times
9. especially - 4 times
10. using - 4 times
Execution time: 504.217330 seconds
[1] + Done                    "/usr/bin/gdb" -
vb4aby.1sh"
root@LaraPC:/home/os_project1#
```

```
TERMINAL

Time taken to read from file: 430.954878 seconds
Top 10 most frequent words:
1. vogt - 8 times
2. term - 8 times
3. indicating - 8 times
4. that - 8 times
5. the - 8 times
6. government - 8 times
7. is - 8 times
8. theoretically - 8 times
9. hers - 8 times
10. not - 8 times
Execution time: 461.996463 seconds
[1] + Done                    "/usr/bin/gdb" --
mmwmu4.cr1"
root@LaraPC:/home/os_project1#
```

```
TERMINAL

Time taken to read from file: 434.948891 seconds
Top 10 most frequent words:
1. biochemist - 6 times
2. him - 6 times
3. as - 6 times
4. a - 6 times
5. populist - 6 times
6. whose - 6 times
7. focus - 6 times
8. on - 6 times
9. consensus - 6 times
10. has - 6 times
Execution time: 469.969640 seconds
[1] + Done                    "/usr/bin/gdb" --
5yhutv.yqx"
root@LaraPC:/home/os_project1#
```

Comments on the Differences in Performance

1. **Naive Approach**:
    - The single-process, sequential execution results in the slowest performance due to the lack of parallelism.
    - This approach serves as the baseline for comparison and highlights the inefficiency of non-parallelized methods for large datasets.
2. **Multiprocessing Approach**:
    - Performance improves significantly as the number of processes increases.
    - The reduction in execution time is noticeable up to 8 processes, where file read times decrease substantially.
    - Overhead from process creation and inter-process communication starts to become apparent beyond 4 processes but remains manageable.
3. **Multithreading Approach**:
    - Performance improvements are visible with more threads, although not as dramatic as with multiprocessing.
    - The use of shared memory eliminates the overhead of inter-process communication but introduces synchronization challenges, which may limit scalability.
    - Beyond 4 threads, performance gains diminish due to thread contention and shared memory synchronization overhead.

Conclusion

- **Naive Approach**: Demonstrates the limitations of sequential processing for large datasets.
- **Multiprocessing Approach**: Shows the best performance improvement, with 8 processes achieving the shortest execution time. However, the optimal configuration aligns with the number of available physical cores (4 processes).
- **Multithreading Approach**: Offers moderate improvements, but is more efficient than the naive approach. The best results are observed with 4 threads, matching the number of cores.

For systems with limited memory or requiring minimal synchronization overhead, multiprocessing is the most effective. Multithreading provides an alternative when memory sharing and lower overhead are priorities.